saroj@prep:~$ cat dataart-ai-production-support-handbook.md

INTERVIEW HANDBOOK

AI Production Support Engineer

DataArt · Regulated Banking Environment · AWS-heavy AI/ML Operations

STATUS: **OPERATIONAL** QUESTIONS: **120** MASTERED: **0** (**0%**) LAST_DEPLOY: **2026-08-17**

reset progress

Search questions & answers…

All | 01 Prod Support/ITIL | 02 AWS & AI Platforms | 03 MLOps | 04 Containers | 05 Observability | 06 CI/CD | 07 Data/Integration | 08 Security | 09 Resilience/DR | 10 Model Risk/Reg | 11 Automation/Cost | 12 Scenarios | 13 Behavioral

☐ unreviewed only

HOW TO USE THIS

Search or click a pill to jump/filter. Tick **Mastered** on any card — progress saves automatically and survives closing this file. "Unreviewed only" hides what you've already ticked so late-stage review stays fast.

DATAART, BRIEFLY

DataArt is a 6,000+ person global IT consultancy (est. 1997, NYC HQ) with a genuinely deep financial-services practice — banks, capital markets, payment providers, fraud-detection systems — and an AWS Consulting Partner designation. Their public positioning leans hard on "regulated, AI-enabled, and governed by design," and they talk about an internal AI delivery model ("Artisyn") that embeds governance and human oversight into AI-assisted delivery specifically for regulated clients. Translation: expect them to actually care about the governance/audit/compliance side of this JD, not just treat it as boilerplate.

PATTERN-MATCH WORTH KNOWING

> Your Aug 5 interview for an almost identically-shaped JD (AI/ML Production Support/SRE, banking, MLOps + model monitoring + ITIL + regulatory) turned out in practice to be a general AWS/cloud-architecture and production-support conversation more than an ML-specific one —

> and the recurring probe was "did *you* design this, or did the platform team." Don't under-prepare the AWS/K8s/CI-CD/DR sections below thinking the "AI" framing means it'll stay ML-flavored throughout. Have your **operator-vs-architect** reframe ready (see §13) regardless of how the conversation opens.

HOW THE ANSWERS ARE CALIBRATED

This is a *production support* role, not a data-science role — the JD itself is written that way (monitoring, incident management, secure deployment, DR, audit logging). Every answer below leans on the "I keep AI systems healthy, observable, and compliant" framing rather than "I build models," which matches both the JD's own emphasis and your real strength profile.

01 · FOUNDATIONS

Production Support & ITIL

L2/L3 support, incident/problem/change management, RCA — the operating discipline the whole JD sits on top of.

01 How does L2/L3 support differ for AI/ML services versus a typical web application?

☐ mastered

L2 handles known-error triage against runbooks — endpoint health, restart a stuck deployment, re-run a failed pipeline step, escalate on a defined threshold (e.g., latency > SLA for 5 min, drift score over threshold). L3 owns anything needing platform or model-context depth: diagnosing *why* a model's predictions degraded (data issue vs. code issue vs. genuine concept drift), tuning autoscaling/resource allocation for a model-serving fleet, or working with Data Science on whether a rollback or a hot-fix retrain is the right call. The extra wrinkle versus a normal app: an ML service can be "up" (200 OK, low latency) while silently wrong — L3 for AI systems has to reason about correctness, not just availability.

02 Walk through the incident management lifecycle you'd apply to an AI service outage.

☐ mastered

Detect (alert fires — CloudWatch alarm, drift monitor, synthetic canary) → **Log & classify** (severity, affected model/endpoint, blast radius — one consumer or bank-wide) → **Triage/mitigate** (stabilize first: fail over, roll back, scale up — restore service before root-causing) → **Resolve** (confirm SLIs back in range) → **RCA** (5 Whys / timeline reconstruction) → **Problem record** (so recurrence is tracked and preventive work gets prioritized) → **Post-incident review** (blameless, action items with owners). In a bank this all needs to land in

ServiceNow (or equivalent) with timestamps that satisfy audit — the ticket trail *is* evidence during a regulatory review.

03 A fraud-detection model's precision drops sharply overnight. Walk me through your RCA.

☐ mastered

5 Whys, applied concretely: **Why did precision drop?** → false-positive rate spiked. **Why?** → check whether the input distribution shifted (data drift via PSI/KS test on recent features) before assuming the model regressed. **Why did the distribution shift?** → trace upstream — a new transaction channel launched, a currency field started arriving in a new format, an upstream ETL (Glue) job changed a join. **Why wasn't that caught pre-deployment?** → gap in schema/contract validation between upstream and the feature pipeline. **Why didn't the drift monitor fire earlier?** → threshold too loose, or monitor only running on a stale batch cadence. Each "why" should be evidence-backed (a metric, a log line, a diff) — not guessed — and the fishbone view (data / model / infra / process / people) is useful when the failure isn't a single clean chain.

> **Interview tip:** naming PSI/KS as your first move (data first, model second) signals you know the most common cause of "model got worse" is upstream data, not the model itself — that's a senior-level instinct interviewers listen for.

04 Distinguish incident management from problem management in ITIL terms.

☐ mastered

Incident management is about restoring service fast — the goal is "stop the bleeding," even with a workaround that doesn't fix the underlying cause. Problem management is about eliminating the underlying cause so the incident class stops recurring — it can run over days or weeks, produces a *known error record*, and often results in a change request. In practice: three incidents in a month for the same SageMaker endpoint hitting a memory limit under load is three incident tickets but should trigger *one* problem ticket that ends in a permanent fix (bigger instance type, request batching change, or a code fix) rather than three separate one-off restarts.

05 How would change management/CAB work for deploying a new model version in a bank?

☐ mastered

Standard changes (pre-approved, low-risk, repeatable — e.g., a config-only endpoint scaling change) can go through a lightweight/automated path. A new model version going to production is almost always a **normal change**: a change record with risk assessment, rollback plan, validation evidence (model evaluation metrics vs. the champion, bias/fairness checks if applicable), and — depending on materiality — sign-off from Model Risk/Compliance in addition

to the CAB. Segregation of duties matters here: the engineer who builds the deployment pipeline shouldn't be the same person who approves the change into production. Emergency changes (hot-fix rollback of a broken model) follow an expedited path but still get retroactive review.

06 How would you structure on-call and escalation for AI production support?

☐ mastered

Tiered: Sev-based paging (Sev1/2 pages immediately, Sev3/4 goes to a queue), a primary + secondary on-call rotation to avoid single points of failure, and a documented escalation path that names *who* to pull in beyond infra — Data Science for model-quality questions, Risk/Compliance for anything customer-impacting on a regulated decision (credit, fraud), and a platform/cloud team if the root cause looks infra-wide rather than service-specific. Runbooks should tell the on-call engineer exactly when to stop self-triaging and escalate — e.g., "if drift score exceeds X and precision drops below Y for a live fraud model, page Data Science within 15 minutes regardless of time of day."

07 What SLAs/SLOs would you set for an ML inference API supporting a customer-facing banking workflow?

☐ mastered

Typical shape: **availability** (99.9%+ depending on criticality), **p99 latency** tight enough not to be the bottleneck in the caller's own SLA (often sub-200ms for a synchronous fraud check inline in a payment flow), **error rate** (5xx/timeouts under a threshold), plus ML-specific SLOs that a typical web SLO set misses: **data freshness** (features no older than X minutes), **drift/quality SLO** (e.g., prediction distribution stays within N standard deviations of baseline), and **time-to-detect** for quality degradation, not just downtime. The error budget conversation with the business has to include what happens when the model is technically "up" but its predictions are untrustworthy — that's a different failure mode than a normal service SLO covers.

08 What belongs in a good runbook for an AI production service, and what doesn't?

☐ mastered

Belongs: the alert that triggers it, exact diagnostic commands/dashboard links, decision tree ("if X, do Y; if that doesn't resolve it in N minutes, escalate to Z"), rollback steps with exact commands, and known false-positive patterns (so on-call doesn't chase ghosts). Doesn't belong: vague prose ("investigate the issue"), anything that assumes tribal knowledge not written down elsewhere, or steps that go stale quickly (hardcoded instance IDs, specific model version numbers) — those should be looked up live, not baked into the doc. A runbook that can't be

followed correctly by someone paged at 3 AM who's never touched this specific service isn't done yet.

09 How would you classify severity for "the fraud-detection model just started auto-approving everything"? ☐ mastered

Sev1, immediately — this is a financial and regulatory exposure, not just a technical outage: fraud loss accrues in real time and every approved transaction during the window is a potential audit finding. The right first move isn't diagnosis, it's containment: fail over to the champion/previous model version or a safe-default rule set (even a conservative "flag for manual review" fallback), *then* RCA in parallel. Communication matters as much as the technical fix here — Risk/Compliance and the business need to know the exposure window (start time, end time, approximate transaction count/value affected) as part of the incident record, not as an afterthought.

02 · CLOUD & AI PLATFORMS

AWS Cloud & AI Platforms

SageMaker, EC2, EKS, Lambda, S3, CloudWatch — the JD's primary skill line and the heaviest-weighted section here.

01 Give me the shape of SageMaker as a platform — what are its major pieces? ☐ mastered

Studio — the IDE/notebook environment for data scientists. **Training jobs** — managed, ephemeral compute for model training, spun up and torn down per job. **Pipelines** — orchestrated, versioned workflows chaining preprocessing → training → evaluation → registration. **Model Registry** — versioned models with approval status. **Inference** — four hosting patterns: real-time endpoints (always-on, low-latency), serverless endpoints (scale-to-zero, spiky traffic), asynchronous endpoints (large payloads, minutes-long inference queued), and batch transform (offline scoring of a full dataset). As production support, you live mostly in training-job failures, endpoint health, and pipeline failures — not model-building itself.

02 When would you choose each SageMaker inference option for a banking workload? ☐ mastered

- **Real-time endpoint** — inline fraud scoring during a payment, sub-second SLA, steady traffic. Costs run 24/7.

- **Serverless endpoint** — internal tools, spiky/intermittent traffic (e.g., an analyst querying a credit model a few times an hour) — accept cold-start latency to avoid paying for idle capacity.
- **Async endpoint** — large documents (e.g., scoring a loan application packet with attachments), inference takes seconds-to-minutes, caller doesn't need a synchronous response.
- **Batch transform** — nightly re-scoring of a whole customer portfolio for risk reporting; no live endpoint needed at all.

03 Where does plain EC2 still show up in an ML production stack that's mostly SageMaker/EKS?

☐ mastered

Custom model-serving stacks that don't fit SageMaker's container contract, GPU workloads needing very specific driver/CUDA versions the managed service doesn't expose, bastion/jump hosts for locked-down VPC access, and self-managed components some teams still run outside Kubernetes (a legacy feature-engineering box, a licensing-constrained tool). Production support angle: right-sizing instance families (GPU vs. CPU-optimized), Spot for interruptible training/batch work, On-Demand or Reserved/Savings Plans for anything that must stay up, and patching/AMI lifecycle discipline since these aren't as auto-managed as a SageMaker endpoint.

04 Why would a team run model inference on EKS instead of just using SageMaker endpoints?

☐ mastered

Control and portability: custom networking/sidecar requirements (mTLS, a specific service mesh), fine-grained autoscaling on custom metrics (queue depth, GPU utilization) that's easier to wire up with HPA than SageMaker's native scaling, multi-cloud or hybrid-cloud strategy where the bank doesn't want to be fully locked to one vendor's inference layer, or simply that the platform team already standardized deployment tooling (Helm/ArgoCD) on EKS for everything and wants ML services to follow the same operational pattern as every other microservice rather than living as a special case. Trade-off: you take on more of the plumbing (autoscaling, health checks, canary logic) yourself instead of getting it managed.

05 What's Lambda's role around an ML pipeline, and what are its real limits?

☐ mastered

Glue code, not the model itself: pre/post-processing a request before it hits a SageMaker endpoint, orchestration triggers (S3 object-created event → kick off a retraining Step Functions workflow), lightweight feature transforms, or fronting an endpoint behind API Gateway for auth/throttling. Limits that matter operationally: 15-minute max execution, up to 10GB memory,

cold starts (mitigated with provisioned concurrency for latency-sensitive paths), and it's a poor fit for actually hosting a large model — that's what SageMaker/EKS are for. Watch concurrency limits too — a burst of traffic can hit account-level concurrent-execution caps and start throttling.

06 How would you structure S3 for an ML platform, and what production-support concerns come with it?

☐ mastered

Typical layering: raw/landing zone → curated/feature zone → model-artifacts bucket → inference-logs bucket, each with its own lifecycle and access policy rather than one flat bucket. Concerns you actually own: **versioning** on model-artifact and training-data buckets (so a bad training run or a corrupted upload is recoverable), **lifecycle policies** (move old model versions to Glacier rather than let costs creep), **encryption** (SSE-KMS with customer-managed keys, not just SSE-S3, for anything touching customer data — a bank's compliance team will ask specifically), and **bucket policies/VPC endpoints** so training jobs and endpoints never traverse the public internet to reach S3.

07 What would you actually watch in CloudWatch for a SageMaker endpoint, day to day?

☐ mastered

Built-in endpoint metrics: `ModelLatency`, `OverheadLatency`, `Invocations`, `Invocation4XXErrors` / `5XXErrors`, and instance-level `CPUUtilization` / `MemoryUtilization` / `GPUUtilization` / `DiskUtilization`. Beyond the defaults, you'd typically push custom metrics from the inference container — prediction-score distribution, feature-null-rate, business-relevant proxies like "fraction flagged as fraud this hour" so a sudden spike or flatline is visible before Data Science's drift job even runs. Alarms should be layered: a fast-firing latency/error alarm for immediate paging, and a slower-trending alarm (e.g., 6-hour rolling average shift) for quality-adjacent signals that shouldn't page at 3 AM on a single blip.

08 How does SageMaker endpoint auto-scaling work, and what's a common misconfiguration?

☐ mastered

It's Application Auto Scaling on top of the endpoint's production variant — target-tracking on a metric like `SageMakerVariantInvocationsPerInstance`, with min/max instance counts and cooldown periods. Common misconfiguration: setting the target too tight (scales constantly, thrashes, and each scale-out has cold-start latency for a freshly-loaded model, especially a large one) or leaving min-capacity at a level that can't absorb a traffic burst before scaling catches up — for a latency-sensitive fraud check, under-provisioned min-capacity plus slow scale-out is exactly how you get an SLA breach during a legitimate traffic spike (e.g., a sale event driving

payment volume). Serverless endpoints sidestep this at the cost of cold-start latency on every scale-from-zero.

09 How do you roll out a new model version on a SageMaker endpoint without a big-bang cutover?

☐ mastered

SageMaker supports multiple **production variants** behind one endpoint with weighted traffic splitting — deploy the new model as a second variant at 5–10% traffic, watch latency/error/quality metrics against the incumbent, and shift weight up gradually. For deployment-level safety rather than manual weight-shifting, **SageMaker's built-in deployment guardrails** (canary or linear traffic shifting with CloudWatch-alarm-based auto-rollback) do this automatically — you define the alarm, the rollout pauses or reverts if it fires. Either way the rollback path has to be as tested as the rollout path — "can we revert in under 5 minutes" is usually the actual question a compliance reviewer asks.

10 Is "SageMaker Studio" still the right term to use? What's Unified Studio?

☐ mastered

Worth knowing AWS has been consolidating: **Amazon SageMaker Unified Studio** is AWS's newer single workspace that brings SageMaker (ML/AI), EMR/Glue (data processing), Redshift/Athena (analytics), and Bedrock (generative AI) under one UI, part of positioning "SageMaker" as the umbrella brand for the whole data+AI platform rather than just the ML-specific tool. Classic SageMaker Studio still exists and most day-to-day production-support terminology (endpoints, training jobs, Pipelines, Model Registry) hasn't changed underneath. Fine to use "SageMaker Studio" in conversation — if the interviewer mentions Unified Studio, you'll know it's the platform consolidation, not a different product replacing what you know.

11 Multi-model endpoint vs. multi-container endpoint — what's the difference and when does each matter operationally?

☐ mastered

Multi-model endpoint (MME): many models of the *same framework/container* sharing one fleet of instances, loaded/unloaded from S3 on demand — great for something like "one fraud model per merchant category" where you'd otherwise need dozens of always-on endpoints. Operationally you watch model cache hit rate and per-model load latency on first invocation. **Multi-container endpoint**: several *different* containers (maybe different frameworks entirely, or a pre/post-processing container chained with the model container) invoked as a pipeline behind one endpoint. Production-support-wise, MME is mostly a cost/density story; multi-container is

more of an architecture/dependency story — a failure in one container in the chain has to be diagnosable independently.

12 Where's the cost actually going on a SageMaker-heavy ML platform, and what would you optimize first? ☐ mastered

Usually: over-provisioned always-on real-time endpoints for workloads that are actually spiky (candidate for serverless or scheduled scale-down), oversized instance types chosen "to be safe" rather than right-sized from actual utilization data, GPU instances left running for training experimentation instead of using Spot with checkpointing, and forgotten old endpoints/model versions nobody decommissioned. First move as production support: pull CloudWatch utilization data per endpoint, flag anything under ~20-30% sustained utilization as a right-sizing candidate, and put a lifecycle policy on model artifacts so S3 storage costs don't quietly grow forever.

03 · MLOPS & MODEL MANAGEMENT

MLOps & Model Management

SageMaker Pipelines, MLflow, Model Registry, drift/data-quality monitoring — the section that actually makes this an "AI" role rather than generic SRE.

01 How is MLOps actually different from the DevOps you already know? ☐ mastered

Same core discipline — automation, CI/CD, monitoring, IaC — applied to an artifact that degrades silently. A code deploy either works or throws an error; a model deploy can pass every health check and still be quietly wrong because the world it's scoring changed. So MLOps adds three things DevOps doesn't need: **data versioning** alongside code and artifact versioning, **continuous evaluation** of output quality (not just uptime), and a **retraining loop** as a first-class pipeline, not a one-off event. Everything you already know about pipelines, rollback, and observability still applies — you're extending it to cover data and model quality as additional first-class signals.

02 What does a SageMaker Pipeline actually look like, and what do you own as production support? ☐ mastered

A DAG of steps — typically `ProcessingStep` (feature engineering/validation) → `TrainingStep` → `EvaluationStep` (compute metrics, often gated with a `ConditionStep`) → `RegisterModelStep` (push to Model Registry, usually landing in "Pending Manual Approval") →

optionally a `TransformStep`/deploy step. As production support you're rarely touching the DAG definition itself — you own: pipeline execution failures (a step OOM'd, an upstream data path didn't exist, a container image failed to pull), execution history/audit trail (who ran what, when, with which inputs — this is exactly what a bank's model-risk audit will ask for), and making sure failed executions alert rather than fail silently.

03 Where does MLflow fit if the platform is mostly SageMaker — are they competing or complementary?

☐ mastered

Usually complementary in practice. MLflow Tracking is often used as the framework-agnostic experiment log (params, metrics, artifacts) because data scientists like its lighter-weight UI and it isn't tied to one cloud, while SageMaker handles the managed compute and production hosting. MLflow's Model Registry and SageMaker's Model Registry do overlap in purpose (versioning, stage transitions, approval), so a shop that has both usually treats *one* as the system of record and the other as a mirror/reference — worth clarifying in an interview which one is authoritative here, since supporting a two-registry setup without a clear source of truth is its own operational headache (you don't want a model marked "Production" in one and "Staging" in the other).

> **Interview tip:** asking "which registry is the system of record here?" is a good clarifying question that shows you've thought about the failure mode of having two.

04 Why does a Model Registry matter so much in a regulated environment specifically?

☐ mastered

It's the auditable spine of the whole model lifecycle: every version tied to the exact training job, dataset snapshot, hyperparameters, and evaluation metrics that produced it; explicit approval status (Pending/Approved/Rejected) with who approved and when; and a clean answer to "what's running in production right now, and can we prove it's the version that passed validation." Without it, "which model is live" becomes tribal knowledge, and a regulator asking "show me the approval trail for the model that made this credit decision on March 3rd" turns into an incident of its own. Production support's job is making sure what's *actually deployed* always matches what the registry says is Approved/Production — drift between the two is a finding waiting to happen.

05 Compare SageMaker-native deployment, KServe/Seldon on EKS, and a custom FastAPI container.

☐ mastered

- **SageMaker-native** — least ops overhead, built-in autoscaling/monitoring/deployment guardrails, but you're inside SageMaker's container/runtime contract.

- **KServe/Seldon on EKS** — Kubernetes-native model serving with CRDs for canary/A-B/explainability out of the box, fits a shop that's standardized on K8s for everything and wants ML services operated identically to other microservices; more plumbing to own (the operator itself, autoscaling config, ingress).
- **Custom FastAPI/Flask container** — maximum control, used when the model needs bespoke pre/post-processing or a framework SageMaker's built-in containers don't support well — but you own health checks, graceful shutdown, concurrency handling, and autoscaling logic entirely yourself.

Given this JD lists both SageMaker *and* EKS as required, the real environment is likely a mix — some models on managed endpoints, others on EKS for the reasons in Q04 above.

06 Explain data drift vs. concept drift, and how you'd actually detect each.

☐ mastered

Data drift: the input distribution changes but the true relationship between features and outcome hasn't — e.g., transaction amounts trend higher post-inflation. Detected by comparing recent feature distributions against a training-time baseline: **PSI** (Population Stability Index — rule of thumb, >0.2 is a meaningful shift) or a **KS test** per feature. **Concept drift**: the relationship itself changes — the same feature values now mean something different (a new fraud pattern the model was never trained on). Harder to detect directly since it needs ground-truth labels, which often lag (you don't know a transaction was fraud until a chargeback lands weeks later) — so proxy signals (rising override/appeal rate, rising manual-review escalation) often catch it before the labeled metric does. **SageMaker Model Monitor** automates the data-drift side against a captured baseline on a schedule.

07 What does data-quality monitoring actually check, beyond drift?

☐ mastered

Schema conformance (expected columns present, correct types), completeness (null/missing-value rate within bounds), range/bounds checks (an age field showing 200, a transaction amount showing negative where it shouldn't), and cardinality checks on categoricals (a new, unseen category value showing up). **SageMaker Model Monitor's** data-quality monitor type covers this against a baseline statistics/constraints file; **Deequ** (built on Spark) is the common choice when the check needs to run inside a Glue/EMR ETL job rather than at the endpoint. The production-support job is making sure these checks run on a cadence that catches a bad upstream deploy *before* it's silently degraded a day's worth of predictions, not after.

08 Explain champion/challenger and shadow deployment — how are they different, and when would you use each?

☐ mastered

Shadow deployment: the new model receives a copy of live traffic and scores it, but its output is logged/compared only — never returned to the caller or acted on. Zero customer risk; used to validate a new model's real-world behavior before it can affect anything. **Champion/challenger**: the challenger's predictions are actually served to a (usually small) slice of real traffic and its outcomes matter — closer to a live A/B test. In a bank, shadow mode is almost always the required first step for anything touching credit or fraud decisions — you want weeks of "would this model have done anything differently, and was it better" evidence before a challenger ever gets to influence a real decision.

09 A newly deployed model version is behaving badly. What's your rollback approach?

☐ mastered

Shift traffic weight back to the previous production variant immediately (this should be a single command/pipeline stage, not a manual redeploy from scratch — if it isn't, that's a gap to flag before it's ever needed live). Confirm via metrics that the rollback actually restored expected behavior, not just that traffic moved. Then: mark the bad version "Rejected" in the registry with the reason, open a problem record, and — this is the part regulated environments care about specifically — document the exposure window and what was affected, since "how long was the bad model live and what decisions did it make" is exactly what an incident report needs to answer for Risk/Compliance.

10 How would you A/B test two model versions in production without confounding the result?

☐ mastered

Consistent hashing on a stable key (customer ID, not request ID) so the same entity always lands in the same arm — otherwise a customer flip-flopping between model versions across requests corrupts both the experience and the analysis. Hold the split stable long enough to reach statistical power on the metric that actually matters (fraud caught, false-positive rate) rather than a proxy. Watch for seasonality/time confounds (don't compare a weekday cohort to a weekend cohort). And in banking specifically, get sign-off on the split logic itself before launch — "which customers are being scored by an unapproved challenger model" is a question compliance will ask, and "random 10%" needs to be defensible, not just convenient.

11 What problem does a feature store (e.g., SageMaker Feature Store) actually solve?

☐ mastered

Training/serving skew — the classic MLOps failure where a feature is computed one way in an offline training pipeline (batch, Python/Spark) and slightly differently in the online serving path (real-time, different code, different data source), so the model sees subtly different inputs live

than it did in training. A feature store gives one definition, computed once, with an **offline store** (for training, historical/point-in-time correct) and an **online store** (low-latency lookup at inference time) backed by the same source of truth. Operationally you're watching feature freshness (is the online store lagging the source data) and consistency between the two stores — a stale online store is a data-quality incident even if nothing "broke" in the traditional sense.

12 Scheduled retraining vs. drift-triggered retraining — trade-offs?

☐ mastered

Scheduled (weekly/monthly): predictable cost and validation workload, easy to plan CAB/approval around, but can retrain on a fine model (wasted compute) or miss a fast-moving drift event between scheduled runs. **Drift-triggered**: reacts faster to real degradation, but needs a trustworthy trigger — a noisy drift signal causing frequent unplanned retrains creates its own operational and governance burden (every retrain in a bank likely needs the same evaluation-gate/approval flow as a scheduled one, so "cheap to trigger" isn't the same as "cheap to ship"). Most mature setups do both: a scheduled baseline cadence plus an alert that fast-tracks an out-of-cycle retrain review when drift crosses a hard threshold, rather than fully automating retrain→deploy with no human gate.

13 Where does explainability (SageMaker Clarify / SHAP) intersect with your role as production support?

☐ mastered

You're not building the explanations, but you own that they *exist and are reproducible on demand*: SageMaker Clarify jobs (bias metrics pre- and post-training, SHAP-based feature attribution) run as part of the pipeline and their output is stored alongside the model version in the registry, not generated ad hoc when someone asks. When a customer disputes a declined loan and compliance needs "why did the model decide this," production support's job is making sure that explanation can actually be regenerated for that specific model version and that specific input — which means monitoring that the Clarify step in the pipeline isn't silently failing or being skipped.

14 What's the "three-way versioning problem" in MLOps, and how do you handle it operationally?

☐ mastered

A model's behavior is fully determined by three things that can each change independently: **code** (training/inference logic), **data** (the training dataset snapshot), and the resulting **model artifact**. Version only the code (standard git) and you can still not reproduce a model, because the data changed underneath it. The fix is treating all three as linked, immutable references: data snapshots get their own identifier (S3 versioning + a manifest, or a dedicated tool), the training job records the exact data version and code commit it used, and the registered model entry

stores both. Production support's stake in this: when RCA-ing a bad model, "what exact data and code produced this" needs to be a lookup, not an investigation.

04 · CONTAINERIZATION & ORCHESTRATION

Docker & Kubernetes (EKS)

Kept concise here on purpose — this is your strongest area already; the ML-specific wrinkles are what's worth drilling.

01 What's different about a good Docker image for a model-serving container versus a typical microservice? ☐ mastered

Size and layering matter more: ML base images (CUDA, framework runtimes) are already large, so a multi-stage build that keeps the runtime layer lean and excludes training-only dependencies (no need for a training framework's full toolchain inside the inference image) meaningfully affects pull time and cold-start/scale-out speed. Pin exact framework versions — a silent minor-version bump in an inference library can change numerical output slightly, which is a correctness issue for a model in a way it usually isn't for typical app code. And bake in a lightweight health-check endpoint that actually exercises the loaded model (a fast dummy inference), not just "process is running."

02 Sketch the core Kubernetes objects you'd use to serve a model on EKS. ☐ mastered

`Deployment` for the serving pods with resource requests/limits sized for the model's real memory footprint, a `Service` (ClusterIP internally, fronted by an ALB via Ingress for external/internal-only exposure), `ConfigMap` for non-secret config (model version pointer, batch size), `Secret` for anything sensitive (pulled from Secrets Manager via the CSI driver rather than committed to a manifest), an `HPA` tied to a meaningful scaling signal, and readiness/liveness probes tuned to the model's actual load time. For anything canary/traffic-split, you're usually layering a service mesh or an Ingress controller with weighted routing on top rather than relying on raw Kubernetes primitives alone.

03 What EKS-specific setup does GPU-backed model serving need? ☐ mastered

A separate GPU-backed node group (e.g., `g5` / `p-family` instances) with the NVIDIA device plugin DaemonSet installed so Kubernetes can schedule against `nvidia.com/gpu` as a resource, taints/tolerations so only GPU-requiring pods land there (you don't want a random

CPU workload occupying an expensive GPU node), and cluster-autoscaler or Karpenter configured to actually scale that node group — GPU nodes are slow to provision, so under-provisioning min-capacity here is a worse latency hit than on CPU nodes. For AWS access from the pod (pulling a model from S3, writing inference logs), **IRSA** (IAM Roles for Service Accounts) scopes permissions per-pod rather than giving the whole node an over-broad instance profile.

04 Plain CPU-based HPA is scaling your inference pods too late. What would you change?

☐ mastered

CPU is often a lagging indicator for inference workloads — by the time CPU crosses the threshold, the request queue has already backed up and latency has already spiked. Better signals: request queue depth, in-flight-request count, or p95 latency itself as a custom metric via the Prometheus Adapter or KEDA, so scaling reacts to the thing you actually care about. Also worth pairing with a lower `--horizontal-pod-autoscaler-sync-period` and less conservative scale-up stabilization window (the default scale-up behavior is intentionally cautious) if the workload is genuinely bursty, while keeping scale-*down* conservative so you're not flapping.

05 Why do readiness/liveness probes matter more for a large-model-serving pod than a typical service?

☐ mastered

A large model can take tens of seconds to minutes to load into memory on pod start. A naive liveness probe with a short initial delay will kill the pod for "failing" before the model has even finished loading — creating a crash loop that has nothing to do with an actual problem. Fix: a proper `startupProbe` with generous `failureThreshold` / `periodSeconds` to cover worst-case load time, a readiness probe that only returns healthy once the model is genuinely loaded and warmed (not just "server process started"), and a liveness probe that only kicks in after startup succeeds. Getting this wrong is one of the most common "why do my pods keep restarting" tickets on ML services specifically.

06 A model-serving pod is CrashLoopBackOff right after a new model version deploy. Triage?

☐ mastered

`kubectl describe pod` for the exit code/reason first — `OOMKilled` is the most common culprit when a new model version is simply bigger than the last one and the container's memory limit wasn't bumped to match. `kubectl logs --previous` to see the crash output from before the restart. Check whether the new model artifact actually finished downloading/mounting (a truncated or still-uploading S3 object will crash the load step). If none of that shows it, check for a dependency mismatch — a new model version trained with a newer framework version than what's pinned in the serving image will fail to deserialize. Resource-limit and image-version

mismatches between what CI/CD built and what's actually referenced in the manifest account for most of these in practice.

07 How does GPU scheduling actually work in Kubernetes, and what goes wrong operationally? ☐ mastered

The NVIDIA device plugin advertises GPUs as an extended resource (`nvidia.com/gpu`); pods request whole GPUs in their resource spec (fractional/shared GPU needs a more specialized setup like MPS or time-slicing, since vanilla K8s scheduling is whole-unit). What goes wrong in practice: a pod stuck `Pending` because no node in the GPU node group has capacity and Karpenter/cluster-autoscaler is slow to add one; a "GPU not found" error inside the container because the device plugin DaemonSet isn't running on that node yet; or silent under-utilization because nothing's actually watching per-pod GPU utilization (CloudWatch/DCGM exporter into Prometheus) so an oversized allocation goes unnoticed for months.

05 · MONITORING & OBSERVABILITY

Monitoring & Observability

CloudWatch, CloudTrail, Prometheus, Grafana, OpenTelemetry, Splunk — and the ML-specific signals none of these give you for free.

01 Apply "the three pillars" (metrics/logs/traces) to an ML inference service specifically. ☐ mastered

Metrics: latency/error/throughput as usual, plus model-specific ones — prediction-score distribution, drift score, feature-null rate. **Logs**: structured, and critically for a bank, they need to capture *what input produced what output at what model version* — that's the raw material for both debugging and audit reconstruction, not just error logs. **Traces**: a request's path through pre-processing → feature lookup → model invocation → post-processing, so when latency spikes you know whether it's the feature store, the model itself, or a downstream call, rather than treating "inference" as one opaque span. A fourth pillar worth naming for ML specifically: **evaluation/quality signal** — accuracy-proxy metrics over time, which has no clean equivalent in a normal app.

02 What custom CloudWatch metrics would you push from an inference container that aren't there by default? ☐ mastered

Prediction-score distribution (mean/p50/p99 of the model's output score, so a flatline or sudden shift is visible), per-class prediction rate for a classifier (e.g., "% flagged as fraud this hour" — a business-meaningful proxy that's often the fastest way to notice something's wrong), feature-null/missing rate at inference time, and model-load/warm-up time on scale-out events. Push these via the embedded metric format or the CloudWatch SDK from inside the serving code, with dimensions for model version and endpoint variant, so you can compare champion vs. challenger on the same dashboard rather than needing separate ones.

03 Why does CloudTrail matter specifically for an AI platform in banking, beyond general AWS hygiene?

☐ mastered

It's the immutable record of *who did what to the ML platform, and when* — who approved a model in the registry, who changed an endpoint's production variant weights, who modified an IAM policy that touches training data. For a model-risk audit, "prove this specific person approved this specific model version on this date, and no one changed the endpoint config outside the approved pipeline afterward" is a CloudTrail query, not a guess. Production-support responsibility: make sure CloudTrail is enabled account-wide (not just default-region), logs ship to a centralized, access-controlled bucket with integrity validation, and retention meets the bank's regulatory retention requirement (often years, not the default 90 days in CloudWatch Logs).

04 What would you scrape with Prometheus for an EKS-hosted model service, and give me a PromQL example.

☐ mastered

Standard container/pod metrics via kube-state-metrics and node-exporter, application-level metrics exposed by the serving code itself (request latency histogram, in-flight requests, prediction counters) via a `/metrics` endpoint, and — if on GPU — DCGM exporter for GPU utilization/memory. A p99 latency query over 5-minute windows:

```
histogram_quantile(0.99,
  sum(rate(inference_request_duration_seconds_bucket[5m])) by (le, model_versior
)
```

Grouping by `model_version` lets one Grafana panel show champion vs. challenger latency side by side during a canary rollout, which is usually more useful than a single aggregate number.

05 How would you instrument a Python inference service with OpenTelemetry?

☐ mastered

Auto-instrument the web framework (FastAPI/Flask) for inbound request spans via `opentelemetry-instrumentation-fastapi`, then wrap the actual model-invocation step as its own child span so it's separately visible from request parsing/serialization overhead — that split is what tells you whether a latency regression is "the model got slower" or "something around it did." Propagate trace context through any downstream calls (feature store lookup, a chained multi-container pipeline) so the whole request is one connected trace rather than disjoint fragments. Export via OTLP to whatever collector the platform standardized on (often feeding both a Prometheus-compatible backend for metrics and Grafana Tempo/X-Ray for traces).

06 If Prometheus/CloudWatch already cover metrics, what's Splunk actually for here?

☐ mastered

Log aggregation and search across the whole estate — including non-AWS-native and legacy systems a bank inevitably still has — plus the compliance-search use case CloudWatch Logs Insights is weaker at: an auditor or investigator asking "show me every prediction this model made for customer X in Q2" or "search all inference logs for a specific transaction ID across every service" is exactly Splunk's job. In practice metrics platforms (Prometheus/CloudWatch) answer "is it broken and how broken," while Splunk answers "show me the specific evidence" — production support usually ends up querying both during a real incident, not choosing one.

07 How do you design alerting for an ML service without drowning on-call in noise?

☐ mastered

Separate fast operational alerts (latency/error-rate/availability — page immediately, short evaluation window) from slow quality alerts (drift score, prediction-distribution shift — these should use a longer rolling window and a trend check, not a single-sample threshold, because model-quality signals are naturally noisier than infra metrics and a one-off blip shouldn't wake anyone at 3 AM). Route them differently too: quality alerts often go to a shared channel Data Science watches during business hours rather than paging on-call SRE, unless they cross a severity threshold that implies real customer/regulatory impact. And alert on symptoms the business cares about (SLA breach, fraud-rate anomaly) rather than every underlying cause independently, or you get five alerts for one incident.

08 What SLIs matter for an ML service that a standard web-service SLI set doesn't capture?

☐ mastered

Data freshness — how stale can the features feeding a prediction be before it's untrustworthy. **Prediction-quality/drift SLI** — some bounded measure of how far current predictions have drifted from the validated baseline. **Time-to-detect quality degradation** — not just "is it down"

but "how fast would we notice if it got quietly wrong." **Explainability availability** — can an explanation be generated on demand for a given prediction, which for a credit/lending model is close to a hard requirement, not a nice-to-have. None of these show up in a standard four-golden-signals dashboard, and that's exactly the gap this JD is hiring for.

09 Would you build one dashboard for this service, or several?

☐ mastered

Several, by audience — a single dashboard trying to serve everyone usually serves no one well. An **ops/on-call view**: latency, errors, saturation, alarm status — built for "is something broken right now." A **model-quality view** for Data Science: drift trends, prediction-distribution shifts, evaluation metrics over time. A **risk/compliance view**: approval status of what's live, exposure/audit-relevant counts, DR/backup posture — built for "can we prove this is under control," not for troubleshooting. Building all three off the same underlying metrics store keeps them consistent without forcing one UI to do three jobs.

10 What's the value of synthetic/canary requests against a model endpoint, on top of real-traffic monitoring?

☐ mastered

Real-traffic metrics only tell you something's wrong once real customers are already affected, and during genuinely low-traffic windows (overnight, weekends) a degraded endpoint can go unnoticed simply for lack of volume to trigger a threshold. A scheduled synthetic request with a known input and a known expected output range — run every few minutes regardless of real traffic — catches both availability issues and a specific class of silent failure: the endpoint answers, but with a value clearly outside the expected range for that fixed input, which is a strong early signal of a broken deploy or a corrupted model artifact before any real customer traffic reveals it.

06 · CI/CD & DEVOPS

CI/CD & DevOps for ML

CodePipeline/CodeBuild/CodeDeploy, Jenkins, GitHub Actions — and the approval-gate discipline that makes ML CI/CD different in a bank.

01 What does a model's CI/CD pipeline have that a typical application pipeline doesn't?

☐ mastered

A data-validation stage before training even starts (fail fast if the input data doesn't match expected schema/quality), a model-evaluation gate after training that compares the candidate

against the current champion on held-out data and blocks promotion if it doesn't clear a defined bar, and — in a regulated shop — a human-approval gate that a normal app deploy usually doesn't need at all. The build/test/deploy mechanics (CodeBuild running tests, CodeDeploy/CodePipeline orchestrating stages) are the same tooling you already know; what's new is what runs *inside* those stages.

02 Sketch how CodePipeline/CodeBuild/CodeDeploy would chain together for a SageMaker model release.

☐ mastered

CodePipeline as the orchestrator, source-triggered from the repo (or from a SageMaker Pipeline execution completing / a new registry entry landing). **CodeBuild** stage runs unit tests on the inference code, builds/pushes the serving container to ECR, and can kick off or validate a SageMaker Pipeline execution. A manual-approval action sits between "model registered" and "deploy to production" for anything requiring sign-off. **CodeDeploy** (or a CodeBuild step calling the SageMaker API directly) handles the actual endpoint update — ideally using SageMaker's built-in canary/linear deployment guardrails with a CloudWatch-alarm rollback trigger rather than a flat cutover.

03 Jenkins and GitHub Actions are both listed — why would a bank run both, and where does each tend to show up?

☐ mastered

Jenkins is very often the incumbent in a bank — self-hosted, sits inside the existing network perimeter, integrated with legacy on-prem systems and years of accumulated pipeline logic nobody wants to migrate overnight. GitHub Actions tends to show up on newer, cloud-native workstreams — exactly the kind of greenfield-ish AI/ML platform work this JD describes — where OIDC federation to AWS (no long-lived credentials in CI) is easy to set up and the team doesn't want to inherit Jenkins' operational overhead (patching, plugin management, agent fleet). Realistic expectation for this role: legacy pipelines stay on Jenkins, the newer AI platform tooling is more likely GitHub Actions, and you support both.

04 The JD says "support secure deployment, release, and rollback of AI/ML models." Make that concrete.

☐ mastered

- **Secure deployment** — pipeline runs under a scoped IAM role (not a human's broad credentials), secrets pulled at runtime from Secrets Manager, container images scanned before deploy, deploy action logged to CloudTrail.
- **Release** — traffic-shifted rollout (canary/linear) with automated CloudWatch-alarm rollback, not a flat cutover; release gated on the model's registry approval status, not just "pipeline succeeded."

- **Rollback** — a tested, single-command path back to the last known-good production variant, with the rollback itself logged and traceable to the incident/change record that triggered it.

05 How do approval gates and segregation of duties actually get enforced in the pipeline, not just on paper?

☐ mastered

Technically, not just procedurally: a CodePipeline manual-approval action (or a GitHub Actions `environment` with required reviewers) that mechanically blocks the pipeline until a named approver — who isn't the committer/deployer — clicks approve, with that action logged with identity and timestamp. Branch protection requiring a second reviewer before merge to the branch that triggers production deploys. IAM structured so the identity/role that *can* push code isn't the same identity/role that *can* approve a production promotion. "We have a CAB process" is a much weaker answer in an audit than "the pipeline physically cannot proceed without a second, differently-privileged identity's sign-off."

06 How would GitOps (ArgoCD) apply to model-serving manifests on EKS?

☐ mastered

Same pattern as any GitOps workload: a Helm chart or Kustomize overlay for the model-serving Deployment/Service/HPA lives in a Git repo, parameterized by model version/image tag; CI updates that repo (bumping the tag) once a model passes its evaluation gate and is registry-approved; ArgoCD detects the diff and syncs the cluster to match. The advantage for a regulated environment specifically: the Git history *is* the audit trail of exactly which model version was live in the cluster at any point in time, and ArgoCD's drift detection catches anyone who manually `kubectl edit` 'd something out-of-band — which itself is a control worth mentioning to a compliance-minded interviewer.

07 How should secrets be handled through this pipeline — what's the wrong way you'd flag in a review?

☐ mastered

Wrong: secrets hardcoded in pipeline YAML, baked into a container image layer, or stored as plain CI/CD platform "environment variables" without encryption/access scoping. Right: pulled at runtime from Secrets Manager (or Parameter Store for non-sensitive config) using the pipeline's scoped IAM role, with rotation enabled so a leaked credential has a short shelf life, and access to read a given secret restricted to the specific roles that need it — a training-job role shouldn't be able to read a production database credential it has no reason to touch. GitHub Actions OIDC federation to AWS removes the need for long-lived AWS access keys in CI entirely, which is worth naming as the modern baseline.

08 Canary vs. blue-green for a model release — which fits better here and why?

mastered

Blue-green gives an instant, clean cutover and instant rollback (flip the router back), but you either serve 100% old or 100% new traffic — no gradual signal-gathering, which is risky for a model where "is this actually good" needs real-traffic evidence, not just a passed test suite. **Canary** (progressively shifting weight — SageMaker's built-in deployment guardrails or a service-mesh weighted route on EKS) lets you catch a quality regression on 5-10% of traffic before it's fully live, which matters more for models than for typical code because a model can pass every automated test and still behave badly on real-world data distribution. For anything touching a regulated decision, canary with an automated metric-based rollback trigger is the safer default.

09 How do you keep artifact/model versioning consistent as a release moves through the pipeline?

mastered

Immutable tags at every stage, never `latest`: the model artifact gets a version tied to its training-job ID, the serving container image is tagged with a content hash or build ID (not a mutable branch tag), and the Model Registry entry references both explicitly. The deployed manifest (whether a SageMaker endpoint config or a K8s Deployment) should record exactly which registry version and which image tag it's running, so "what's live right now" is always a direct lookup rather requiring cross-referencing multiple systems and hoping they agree.

07 · DATA & INTEGRATION

Data & Integration

Glue, Kinesis, Kafka, EventBridge, REST APIs, RDS/DynamoDB/Aurora — the plumbing feeding features in and predictions out.

01 Where does AWS Glue sit in an ML pipeline, concretely?

mastered

The ETL layer that turns raw source data (transaction logs, customer records) into the curated feature tables a SageMaker Processing/Training step consumes — Glue jobs (Spark under the hood) doing joins, aggregations, and the schema/quality validation checks from the MLOps section. The **Glue Data Catalog** is the metadata layer everything else points at — Athena queries, SageMaker Processing jobs reading from S3 via a catalog table, and it's also where a data-lineage story starts (which table fed which pipeline). Production-support angle: Glue job

failures (schema mismatch, a source partition that didn't land on time) are usually the actual root cause behind "the model's inputs look wrong," not the model itself.

02 Kinesis vs. Kafka (MSK) — when would a bank use each for feeding an ML pipeline?

☐ mastered

Kinesis Data Streams — fully managed, less operational overhead, integrates natively with Lambda/Firehose/other AWS services, a good fit when the team wants to stay AWS-native and doesn't need Kafka's broader ecosystem. **Kafka (via MSK)** — chosen when the bank already has Kafka elsewhere (very common — core banking/payments event backbones are frequently Kafka already), needs its richer ecosystem (Kafka Streams, Connect, cross-team topic sharing at a scale Kinesis shard limits make awkward), or needs multi-consumer-group fan-out patterns. Given this JD's own note explicitly adds Kafka alongside Kinesis, the real environment likely already has Kafka as the event backbone and Kinesis/EventBridge fill AWS-native gaps around it — worth asking directly in the interview which is the system of record.

03 Give a concrete EventBridge use case in this kind of platform.

☐ mastered

Event-driven orchestration glue rather than high-throughput streaming: an S3 `PutObject` event on the training-data bucket triggering an EventBridge rule that kicks off a SageMaker Pipeline execution; a SageMaker Model Registry state-change event (model approved) triggering a downstream Step Functions workflow that starts the deployment pipeline; or a scheduled rule (EventBridge Scheduler) firing a nightly batch-scoring job. It's the "when X happens, start Y" layer connecting otherwise-decoupled AWS services without needing a poller or a cron job babysitting things.

04 What matters when designing (or reviewing) a REST API contract for model inference?

☐ mastered

Explicit **versioning** in the path or header (`/v2/score`) so a model-version change doesn't silently break a caller expecting the old response shape. **Idempotency** for anything that might be retried (a payment-adjacent caller retrying on timeout shouldn't cause a double-scored/double-charged side effect) via an idempotency key. Clear, typed error responses distinguishing "your input was invalid" (4xx) from "we failed to score this" (5xx) so callers can build correct retry logic. And returning enough metadata in the response — model version, a request/trace ID — that a downstream issue can be traced back to exactly which model made which decision, which is both an operational and an audit need.

05 How does the data/integration layer differ between batch and real-time inference?

mastered

Batch: Glue/EMR reads a full dataset from S3, SageMaker batch transform scores it, results land back in S3 or a warehouse — throughput-optimized, latency doesn't matter, failure recovery is "re-run the job." **Real-time**: the feature pipeline has to serve fresh values in milliseconds (online feature store, or a low-latency cache in front of a database), the request path is synchronous and every hop's latency compounds into the SLA, and failure handling needs to be graceful in-request (a timeout on a feature lookup needs a defined fallback, not just an error bubbling up to the caller). Production support tends to live mostly in the real-time path's failure modes, since batch failures are usually just "job failed, re-run" with much more forgiving time pressure.

06 RDS, DynamoDB, or Aurora — how would you choose for ML-adjacent metadata storage?

mastered

DynamoDB — online feature-store-style lookups needing single-digit-millisecond latency at high request volume, simple key-value/document access patterns (a customer ID → current feature vector). **RDS/Aurora (relational)** — anything needing complex joins/transactions/referential integrity: model metadata, approval-workflow state, prediction-audit logs that need to be queried relationally ("show me all predictions where confidence < 0.6 for model version X"). **Aurora specifically over vanilla RDS** when the workload needs Aurora's higher throughput/read-replica scaling and its cleaner story for cross-region DR — see next question.

07 The JD's closing note specifically calls out AuroraDB rather than generic RDS — why would a bank prefer it?

mastered

Aurora's storage layer is decoupled from compute and replicated across 3 AZs automatically (6 copies of data), giving faster failover (typically under 30 seconds) than a standard Multi-AZ RDS instance's replica promotion. **Aurora Global Database** gives sub-second cross-region replication with a documented, testable RTO/RPO story — directly relevant to the DR section of this JD. Up to 15 read replicas (vs. 5 for standard RDS engines) supports read-heavy audit/reporting queries without touching the primary. For a bank's model-metadata/prediction-audit store, that combination — fast failover, provable cross-region DR, read scaling for compliance reporting — is usually the actual justification, not just "it's AWS's preferred flavor."

08 How would you design a feature pipeline that needs both streaming and batch sources?

mastered

This is the classic "Lambda architecture"-flavored problem in a feature-engineering context: a streaming path (Kafka/Kinesis → a stream processor or Lambda) computes fast-moving features (last-5-minutes transaction velocity) and writes to the online feature store for low-latency serving, while a batch path (Glue, scheduled) computes slower-changing aggregates (30-day average spend) and backfills both online and offline stores. The operational risk to watch: the two paths computing the "same" feature slightly differently (different windowing logic, different null-handling) — which is exactly the training/serving skew problem a feature store exists to prevent, so both paths should share one feature-definition source rather than being independently implemented.

09 Walk through an API Gateway → Lambda → SageMaker endpoint request path, and where it can fail. ☐ mastered

API Gateway handles auth (IAM/Cognito/API key), throttling, and request validation; Lambda does pre-processing (feature assembly, input transformation) and calls `InvokeEndpoint` against SageMaker; the response is post-processed and returned. Failure points worth knowing cold: API Gateway's own default throttle limits (a traffic spike can get throttled before it even reaches Lambda), Lambda's 15-minute timeout and cold-start latency compounding with SageMaker's own inference latency, and IAM — the Lambda execution role needs explicit `sagemaker:InvokeEndpoint` permission scoped to that specific endpoint, which is a very common "works in dev, 403s in prod" gap when a new endpoint is added but the role's resource ARN wasn't updated.

10 What does "data lineage" mean in practice, and how would you implement it here? ☐ mastered

Being able to answer, for any prediction, the full chain: which source system the input data came from, which Glue job/transformation touched it, which feature-store version served it, which model version scored it, and who approved that model — end to end, not just "the model version." Implementation is mostly disciplined tagging and metadata, not one magic tool: Glue Data Catalog table lineage, consistent run/version IDs threaded through every pipeline stage (the training job records its input data version; the registry entry records the training job ID; the inference log records the model version), and structured logs that make the chain queryable rather than requiring someone to manually reconstruct it during an audit.

08 · SECURITY & IDENTITY

Security & Identity

IAM, KMS, Secrets Manager, VPC security — non-negotiable depth for a regulated banking environment.

01 What does least-privilege IAM actually look like for an ML platform's various roles?

mastered

Separate, narrowly-scoped roles per function rather than one broad "SageMaker admin" role everyone shares: a **training-job execution role** that can read specific training-data S3 prefixes and write to a specific artifacts prefix, nothing else; a **pipeline-orchestration role** that can start executions but not modify IAM itself; an **endpoint-invocation role** for calling services scoped to `sagemaker:InvokeEndpoint` on named endpoint ARNs only, not `*`. Cross-account access (common when the AI platform account is separate from the core-banking account) should go through assumed roles with explicit trust policies, not shared long-lived keys. Periodic access review — actually pruning unused permissions, not just granting and forgetting — is the part that tends to slip without an owner.

02 Where does KMS show up across the ML pipeline, and why does "customer-managed key" matter to a bank?

mastered

Encrypting training data and model artifacts at rest in S3, EBS volumes attached to training instances, and SageMaker endpoint storage — all should specify a KMS key rather than relying on defaults. **Customer-managed keys (CMKs)** over AWS-managed keys matter because the bank controls the key policy directly (exactly which roles can use it), gets a full CloudTrail audit trail of every encrypt/decrypt use of that specific key, and can enforce key rotation and revocation on their own schedule — an AWS-managed key gives none of that granular control, which is usually a hard compliance requirement rather than a nice-to-have in banking. Also worth knowing: KMS key policies are a common source of "access denied" tickets when a new role needs to use an existing CMK and its key policy wasn't updated to grant it.

03 What's the discipline around Secrets Manager you'd expect to enforce as production support?

mastered

Automatic rotation enabled wherever the target supports it (RDS/Aurora credentials rotate natively with a Lambda rotation function), secrets read at runtime via the SDK/IRSA rather than injected as plaintext environment variables baked into a container image, and resource-based policies scoping which specific roles can read which specific secret. Operationally, rotation failures are a real class of incident worth knowing — if the rotation Lambda fails silently, the secret can end up in an inconsistent state where the old credential still works but the new one

hasn't propagated everywhere, so monitoring the rotation function itself (not just assuming "rotation enabled" means "rotation working") is part of the job.

04 How would you run SageMaker training/inference inside a VPC with no internet access?

☐ mastered

Launch training jobs and endpoints with **VPC config** specifying private subnets and security groups, and enable SageMaker's **network isolation** flag so the container literally cannot make outbound network calls at all (not even to other AWS services) unless explicitly routed. Since there's no NAT/internet route, every AWS service the job needs — S3, ECR (to pull the container image), CloudWatch Logs — needs a **VPC endpoint** (interface or gateway endpoint) in that VPC, or the job fails to even start. This is one of the most common "why won't my training job start" tickets in a locked-down banking VPC — usually a missing VPC endpoint for ECR or S3, not a code problem.

05 What's SageMaker's network isolation flag actually protecting against?

☐ mastered

Data exfiltration and untrusted-code risk from training jobs specifically — a training job runs arbitrary code (whatever the data scientist's container does), and without isolation that code could in principle make an outbound call to anywhere. With network isolation enabled, the container can't reach the internet or even other AWS services except through the VPC endpoints you've explicitly provisioned, so even a compromised or malicious dependency in the training code has nowhere to send data. It's a meaningful defense-in-depth control specifically because training jobs are less scrutinized line-by-line than production inference code usually is.

06 How would you expose an inference API to internal bank systems only — never the public internet?

☐ mastered

A **private API Gateway** (accessible only via VPC endpoint, not a public URL) or an internal-facing ALB in front of the EKS/SageMaker service, both living in private subnets with security groups restricting inbound to only the specific consumer CIDR ranges/security groups that legitimately need access. No public DNS record, no internet gateway route for that subnet. For cross-VPC or cross-account internal consumers, PrivateLink is the standard pattern rather than VPC peering with broad routes — narrower blast radius if a consuming account is ever compromised, since PrivateLink only exposes the specific endpoint service, not the whole VPC's network.

07 IRSA vs. EC2 instance profiles for granting AWS access — why does the distinction matter on EKS?

mastered

An instance profile grants permissions at the *node* level — every pod scheduled on that node inherits the same AWS permissions, which is a broad blast radius on a shared cluster (a compromised or misconfigured pod for one team could reach another team's S3 data). **IRSA** (IAM Roles for Service Accounts) scopes permissions per-*pod* via a Kubernetes service account annotated with a role ARN and an OIDC trust relationship — so the model-serving pod gets exactly the S3/KMS/SageMaker permissions it needs and nothing the node itself, or other pods on it, can also use. For a multi-tenant EKS cluster running several banking workloads, IRSA (or the newer EKS Pod Identity) is close to a hard requirement, not a best-practice nicety.

08 Trace encryption end-to-end across the ML pipeline — where's it at rest, and where's it in transit?

mastered

At rest: raw/curated data and model artifacts in S3 (KMS/SSE-KMS), EBS volumes on training instances, database storage for feature/metadata stores (Aurora/DynamoDB encryption at rest), and CloudWatch/Splunk log storage. **In transit**: TLS between every hop — client to API Gateway, API Gateway to Lambda/backend, Lambda to SageMaker endpoint (SageMaker enforces TLS on the invoke API), inter-pod traffic on EKS if the compliance bar requires mTLS via a service mesh (common for a bank), and replication traffic for cross-region DR. The audit-friendly answer isn't just "we use TLS/KMS" — it's being able to point to where each hop is enforced and prove nothing falls back to plaintext under any failure mode.

09 · RESILIENCE, BACKUP & DR/BCP

Resilience, Backup & DR/BCP

AWS Backup, Cross-Region Replication, Multi-AZ/Multi-Region — proving the AI platform survives a bad day, not just a bad request.

01 Does an AI inference service really need multi-region, or does Multi-AZ cover it?

mastered

Depends on the service's criticality tier, and it's worth pushing back on "just go multi-region" as a default. **Multi-AZ** covers the vast majority of real failure scenarios (an AZ going down, a single data-center issue) with much lower cost and complexity, and is usually sufficient for internal or non-customer-critical models. **Multi-region** is justified when the service is genuinely critical-path for a regulated customer-facing flow (a fraud check blocking every card transaction) where a full

AWS region outage — rare, but it happens — cannot mean the bank stops processing payments. The honest answer in an interview: name the tiering criteria, don't just default to "multi-region everything," since over-engineering DR is also a cost/complexity finding a good architect gets pushed on.

02 What would you actually put under AWS Backup for an ML platform?

☐ mastered

Model artifacts and training-data snapshots in S3 (via S3 backup support), Aurora/RDS databases holding model metadata and prediction-audit logs, EBS volumes for anything stateful outside managed services, and — often overlooked — the *pipeline definitions themselves* (SageMaker Pipeline configs, Model Registry contents) since losing the ability to reconstruct "what pipeline produced this model" is as bad as losing the model artifact for audit purposes. Backup plans should be tagged/scoped by resource so retention matches the regulatory requirement for that data class rather than one blanket policy for everything.

03 Set up Cross-Region Replication for the model-artifacts S3 bucket — what do you need to get right?

☐ mastered

Versioning must be enabled on both source and destination buckets (CRR requires it), an IAM role scoped specifically for the replication action, and — for a bank — replicating the KMS-encrypted objects correctly means either using the same CMK with a cross-region key policy or (more commonly) a separate CMK in the destination region with replication configured to re-encrypt under it. Easy to miss: CRR is not retroactive by default — existing objects at the time you enable it don't automatically replicate unless you run S3 Batch Replication for the backlog. And replication doesn't happen instantly — factor actual replication lag into your RPO calculation rather than assuming it's zero.

04 How would you set RTO/RPO for a fraud-detection model service, and who decides the number?

☐ mastered

The business/risk side sets the target based on financial and regulatory exposure, not engineering — your job is telling them what's achievable at what cost, then building to the agreed number. For a live fraud check inline in payment processing, a realistic ask is often RTO in minutes (a hard outage stopping fraud screening might force the bank to either halt transactions or fail open to a conservative rule-based fallback — both have real cost) and RPO near-zero for the model/config state (you can't lose track of which model version was authoritative) while accepting some feature-freshness RPO is naturally non-zero given streaming

lag. The number isn't just picked — it should be validated by an actual DR test, not assumed from the architecture diagram.

05 Structure a DR runbook for this AI service. ☐ mastered

Trigger criteria (what confirms this is a real DR event, not a transient blip — avoids a costly false-alarm failover), the failover sequence itself (promote the standby Aurora Global Database region, redirect Route53 — usually a failover routing policy with health checks rather than a manual DNS change, repoint the SageMaker endpoint traffic or spin up the standby-region endpoint from the replicated model artifact), validation steps to confirm the failover actually works end-to-end (not just "DNS changed" but "a real inference request in the new region returns a correct result"), and a documented failback procedure once the primary region recovers — failback is often the step teams forget to rehearse and get wrong under pressure.

06 What would a DR game day for an AI workload specifically test, beyond a typical app failover drill? ☐ mastered

Everything a normal failover drill tests (routing, database promotion, RTO measured against target) plus AI-specific checks: does the standby region actually have the *correct, currently-approved* model version loaded (not a stale replica), does the drift/monitoring pipeline come back up in the failover region or does it silently stop watching, and — for a bank — does the audit-logging path (CloudTrail, prediction logs) keep recording correctly through the failover so there's no gap in the compliance record during the exact window regulators would most want visibility into. A DR drill that restores service but loses monitoring coverage during the transition has "fixed" the wrong problem.

07 Your SageMaker endpoint's region is having a genuine AWS outage. Walk me through your response. ☐ mastered

Confirm it's actually an AWS-side regional issue (AWS Service Health Dashboard / Personal Health Dashboard) rather than something in your own control plane — misdiagnosing this wastes the most valuable early minutes. If multi-region DR is provisioned for this service, execute the documented failover runbook rather than improvising. If it isn't (a lower-tier service that was accepted as Multi-AZ-only), the honest move is invoking the pre-agreed degraded-mode fallback — often a conservative rule-based decision path rather than "wait it out" — because in banking, "the model is unavailable" quietly becoming "we skip fraud screening until AWS recovers" is not a decision an SRE should make unilaterally in the moment; it needs a pre-approved fallback plan, made calmly, not invented live.

08 How often should backup/restore actually be tested, and what evidence does a regulator want to see?

mastered

A backup nobody has restored from is a hope, not a control — cadence depends on the bank's own policy but quarterly restore tests for critical systems is a common baseline, with a full DR failover drill at least annually. What a regulator wants isn't "we have backups enabled" — it's a dated test record: what was restored, how long it took against the stated RTO, whether the restored data was verified correct, and who signed off. Production support's job is making sure that evidence trail exists automatically (test run logged, results captured) rather than needing to be manually reconstructed the week before an audit.

10 · MODEL RISK & REGULATORY COMPLIANCE

Model Risk Management & Regulatory Compliance

GDPR and "model risk frameworks" as the JD names them — plus the India-specific RBI framework, since DataArt's financial-services clients span exactly this territory.

01 What is Model Risk Management, and what's the "three lines of defense" model?

mastered

MRM is the discipline of identifying, measuring, and controlling the risk that a model is wrong, misused, or poorly understood — with real financial and regulatory consequences when it fails. **SR 11-7** (US Federal Reserve/OCC guidance, 2011) is the widely-cited reference framework globally, even outside the US, because it's the most mature articulation of the idea: independent model validation separate from the team that built the model, ongoing performance monitoring, and clear documentation. **Three lines of defense**: 1st line = the team building/operating the model (that's you); 2nd line = independent risk/model-validation function that challenges and approves; 3rd line = internal audit checking that the first two lines are actually doing their job. Production support sits squarely in the 1st line — you're not the approver, you're the one whose operational evidence the 2nd and 3rd lines rely on.

02 What does GDPR actually require of an ML system in banking, operationally?

mastered

Right to explanation for automated decisions with legal/significant effect (a declined loan) — ties directly to the explainability tooling in §3. **Data minimization** — only the features genuinely necessary for the decision, not "collect everything, might be useful someday," which affects what a feature pipeline is even allowed to ingest. **Right to erasure** creates a genuinely hard

operational problem for ML specifically: deleting a customer's data from a source database is straightforward, but if that data was already used to *train* a model, the model itself has "learned" from it — full compliance can require retraining without that data, which production support needs a real process for, not just deleting a database row and calling it done.

03 What's the RBI's FREE-AI framework, and why would it matter for a role like this?

☐ mastered

The Reserve Bank of India released FREE-AI (Framework for Responsible and Ethical Enablement of AI) on August 13, 2025, from a committee chaired by Prof. Pushpak Bhattacharyya (IIT Bombay) — seven guiding principles ("sutras"), six strategic pillars, and 26 actionable recommendations covering governance, consumer protection, infrastructure, and assurance for AI use in India's banks/NBFCs/fintechs. Practically relevant here: it pushes for institutional AI governance committees, board-approved AI policies, and baseline gap assessments against the framework — exactly the kind of "governance embedded by design" positioning DataArt's own marketing leans on. Worth knowing even if this specific role serves a non-Indian client, since it signals the direction global banking AI regulation is heading and DataArt likely has India-based clients directly subject to it.

04 Is GDPR even the relevant law, given you're based in India?

☐ mastered

GDPR applies if the client's customers are in the EU — plausible for a DataArt banking client, but not guaranteed. India's own **Digital Personal Data Protection Act, 2023 (DPDPA)** is the more directly relevant law for an India-processed workload and shares GDPR's spirit (consent, purpose limitation, data-principal rights) without being identical in mechanics. The safe, accurate answer in an interview: name both, and note the actual applicable regime depends on whose customer data is being processed and where — which is exactly the kind of question worth asking the interviewer directly if the JD doesn't specify a client, rather than assuming.

05 What does audit logging need to capture so a regulator can reconstruct "why did the model decide this"?

☐ mastered

For a specific prediction: the exact input features (or a reference to their exact values at that point in time), the model version and its registry approval record, the raw output/score and the resulting decision, a timestamp, and — for anything with an explainability requirement — enough to regenerate the SHAP/Clarify explanation for that specific input against that specific model version. For the platform itself: who deployed what, when, with what approval trail (CloudTrail), and whether the endpoint that served this prediction matched what the registry said was "Production" at that moment. The test to hold every logging decision to: could you answer "why

did the model decide this, and was the right process followed" a year later, from logs alone, with no one who remembers the incident still on the team.

06 What's a "model card" or validation report, and whose job is it to keep current?

mastered

A model card is a structured document per model version: intended use and known limitations, training data description, evaluation metrics (including fairness/bias metrics across relevant subgroups), and approval status. Data Science usually authors the content, but production support has real skin in the game: making sure the model card that's referenced actually matches what's deployed (a stale model card describing a model version that's no longer live is a finding), and that the validation report's stated operational assumptions (expected traffic pattern, expected input distribution) are things you're actually monitoring for violation — a model validated assuming stable daily volume that then sees 10x traffic on a sale event is operating outside its validated envelope, and someone needs to know that's happening.

07 Beyond the SHAP/Clarify mechanics, why do regulators specifically care about explainability for credit/fraud models?

mastered

"Adverse action" reasoning — in many jurisdictions, a customer who's declined credit or flagged for fraud has a right to a meaningful reason, not "the algorithm said no." A black-box model that can't produce a defensible, specific explanation for an individual decision creates real legal exposure, independent of how accurate it is in aggregate. It's also a fairness/discrimination control: without per-decision explainability, it's much harder to detect that a model has learned a proxy for a protected characteristic it was never explicitly given. As production support, your stake is operational: the explanation-generation path (whatever produces that per-decision reasoning) needs the same uptime/monitoring discipline as the model itself — a "the model is up but explanations are broken" state is still a compliance incident.

08 Compliance hands you an audit request: "prove no restricted data was used to train this production model." What do you actually do?

mastered

Pull the exact training-job record from the Model Registry for that model version — it should point to an exact, versioned dataset snapshot (this is the three-way-versioning discipline from §3 paying off). Cross-reference that dataset's schema/lineage in the Glue Data Catalog against the list of restricted fields Compliance is asking about — confirming the field either was never in the source table, or was explicitly excluded in the feature-engineering step, with the Glue job logic as evidence. If the lineage trail can't answer this cleanly and quickly, that itself is the finding worth

raising proactively — "we can't currently prove this in under an hour" is a gap to fix before an actual regulator asks, not just for this one audit request.

09 Where exactly does an "AI Production Support Engineer" sit in the three-lines-of-defense model, and why does that matter for how you answer interview questions? ☐ mastered

Squarely 1st line — you operate the control, you don't independently validate it. That's actually a useful frame to state explicitly in the interview: it means your job isn't deciding whether a model is fair or accurate enough (that's Data Science + 2nd-line Model Validation), it's making sure whatever the 2nd line approved is what's actually running, that deviations are caught fast, and that the evidence trail the 2nd and 3rd lines need is always available and trustworthy. Framing your own answers this way — "I make sure the approved thing is the running thing, and I can prove it" — reads as someone who understands governance structure, not just tooling.

11 · AUTOMATION, COST & NICE-TO-HAVES

Automation, Cost Optimization & Nice-to-Haves

Where you'd actually spend effort improving the platform, not just keeping it running.

01 Where would you look first for automation opportunities on a platform like this? ☐ mastered

Anywhere a human is currently the trigger for something a metric could trigger instead, with a human still in the approval loop where governance requires it: auto-rollback on a canary deployment when a CloudWatch alarm fires (rather than someone watching a dashboard and manually reverting), a fast-tracked out-of-cycle retrain review triggered by a drift-threshold breach rather than waiting for someone to notice, and self-healing runbook automation for known-safe remediations (a stuck endpoint update that always resolves with a specific restart sequence shouldn't need a human to type the same commands every time). The line to hold: automate detection and remediation-of-known-issues aggressively; keep a human gate on anything that changes what model is authoritative in production, since that's a governance decision, not just an operational one.

02 What does DevSecOps look like applied specifically to an ML pipeline, versus a normal app pipeline? ☐ mastered

Everything you already do for a normal pipeline — container image scanning (Trivy/ECR scanning) before deploy, dependency/SAST scanning on the codebase, secrets scanning on commits — plus ML-specific additions: scanning the training-data source for the restricted-field problem from §10 *before* training even runs (cheaper to catch there than after a model's already trained on it), and treating the model artifact itself as a supply-chain object worth integrity-checking (a hash mismatch on a model artifact between what the registry says and what's actually loaded is a security-relevant event, not just an operational one — it could mean tampering, not just a bad deploy).

03 What's operationally different about supporting a fraud-detection or credit-risk model versus a generic ML service? ☐ mastered

Stakes and speed both go up. Latency SLAs are tighter because these models usually sit inline in a real-time transaction path, not behind it. Severity classification skews higher by default — a bug that would be Sev3 on an internal recommendation model is plausibly Sev1 here because it directly touches money movement or a regulated decision. Regulatory scrutiny is heavier and more specific (fair-lending laws for credit, AML/sanctions obligations for fraud) so the explainability/audit requirements from §10 aren't optional extras, they're core to the service being allowed to run at all. And false positives have a real customer-trust cost (legitimate transactions blocked) that has to be watched as carefully as false negatives (fraud getting through) — it's a two-sided metric, not just "catch more fraud."

04 Do AWS/MLOps certifications actually matter for this role, and how would you talk about yours honestly? ☐ mastered

They signal structured knowledge, not hands-on depth by themselves — most interviewers know this, and overselling a cert as equivalent to production experience tends to backfire under follow-up questions. Your honest framing: AZ-104 (Azure administration) and especially SC-300/SC-100 (identity and security architecture) map directly onto this JD's IAM/KMS/Secrets Manager/VPC-security section even though they're Azure-branded — identity governance, least-privilege design, and security architecture thinking transfer across clouds far more than service-specific syntax does. Pair that with your self-assessed 6+ years hands-on AWS — the certs demonstrate you formalize what you already do, not that you're starting from a textbook.

05 Give a quick, honest answer on your familiarity with "AI governance frameworks and explainability tools" as a nice-to-have. ☐ mastered

Frame it as operational familiarity rather than deep hands-on tooling expertise, and be specific about what that means: you understand what SageMaker Clarify and SHAP-based explainability

are for and why regulators require them (covered in depth in §3 and §10), you understand the governance-frameworks landscape (SR 11-7, GDPR, RBI FREE-AI) well enough to know what evidence a production system needs to generate, and you're comfortable being the operational owner making sure those tools run reliably and their outputs are trustworthy — without claiming to be the one designing the fairness metrics or governance policy itself. That's an accurate, defensible answer that matches what the JD is actually asking for from this specific role.

12 · SCENARIO BANK

Production Troubleshooting Scenarios

These usually come as "walk me through what you'd do" — lead with containment, then diagnosis, then prevention. Structure matters as much as the technical answer.

SCENARIO

☐ mastered

You're paged at 2 AM: p99 latency on the fraud-scoring endpoint has tripled.

First: check whether it's a traffic-volume story (legitimate spike the autoscaler hasn't caught up to yet) versus a genuine degradation — CloudWatch invocation count alongside latency tells you which. **Contain:** if it's autoscaling lag, manually bump instance count/min-capacity rather than waiting for the scaling policy to catch up. If it's not traffic-driven, check for a resource-exhaustion signal (memory/CPU near limit — possibly a memory leak from a recent deploy) and consider a fast rollback to the previous known-good variant while you investigate rather than debugging live on a degraded production path. **Then diagnose** with traces to find which hop (feature lookup vs. model inference itself) actually regressed. **Escalate** only if it's still unresolved past your runbook's defined threshold — don't sit on it solo past that point.

SCENARIO

☐ mastered

Your drift monitor fires: model accuracy proxy dropped 15% in the last 6 hours.

Don't roll back reflexively — first determine whether this is data drift (upstream distribution shift) or a genuine model/infra problem. Pull PSI/KS-test results on recent feature distributions against baseline. Check whether a known event correlates (a new product launch, an upstream schema change, a batch job that ran with bad data). If the drift is real and upstream, the fix is a data-pipeline fix, not a model rollback — rolling back the model wouldn't even help if the input data itself is the problem. If you can't find an upstream cause and the model's own behavior on

stable inputs has changed, that's when rollback-while-investigating is the right containment move. **Loop in Data Science** either way — this is a joint call, not a unilateral SRE decision, especially for anything touching a regulated outcome.

SCENARIO

mastered

A canary deployment is clearly degrading key metrics, but the automated rollback didn't trigger.

Immediately — manually shift traffic weight back to the previous variant; don't wait to understand why automation failed while customers are on the bad path. **Then** figure out why the alarm-based rollback didn't fire: most commonly, the CloudWatch alarm's evaluation window/threshold was tuned too loosely for how the metric actually degraded (a real quality drop that doesn't cross a raw error-rate threshold, for instance), or the deployment guardrail wasn't actually wired to the alarm you thought it was. **Follow-up** that matters as much as the fix: this is a gap in your safety net that needs a problem record — if it failed silently once, it'll fail silently again on the next deploy unless the alarm/threshold gets fixed and, ideally, tested against a deliberately-bad synthetic deploy before you trust it again.

SCENARIO

mastered

A SageMaker endpoint update has been stuck in "Updating" for 40 minutes.

Check the endpoint's event history/CloudWatch Logs for the update operation — common causes: the new container failed its internal health check on the new instances (so SageMaker won't cut over and is stuck waiting), a capacity issue in the target instance type/AZ, or an IAM permission gap the execution role hit only on this specific update. The endpoint should still be serving traffic on the *old* variant during this — confirm that's actually true (check live invocation success rate) so you know whether this is "annoying but traffic is fine" or "traffic is degraded too." If it's genuinely stuck past a reasonable window, cancel the update rather than let it hang indefinitely, fix the underlying cause (usually the new container's health check or a resource limit), and redeploy.

SCENARIO

mastered

Predictions look consistently off, and you trace it to a Glue ETL job that's been silently failing for two days.

This is a monitoring gap as much as a pipeline failure — a job silently failing for two days means nothing was watching job success/failure status with alerting attached, which is the first thing to fix regardless of this specific incident. Immediate containment: identify exactly which feature(s) went stale and for how long, and whether predictions served during that window should be flagged/re-evaluated (this becomes an incident-scope question for Risk/Compliance if any of those predictions affected a customer decision). Fix the job, backfill the missed data if the source still has it, and verify the feature store actually reflects fresh data before declaring resolved. **Prevention:** a CloudWatch alarm/EventBridge rule on Glue job failure status, not reliance on someone noticing.

SCENARIO

☐ mastered

SageMaker Model Monitor flags a data-quality violation. Real issue, or false positive?

Pull the actual constraint violation report — Model Monitor tells you exactly which feature and which check (a null-rate threshold, a type mismatch, a range violation) triggered it, so start there rather than guessing. Cross-check against anything that changed recently: an upstream schema change, a new data source, or a genuinely unusual-but-valid event (a holiday causing an unusual transaction pattern that's real, not broken). If it's a genuine data problem, treat it like the Glue-failure scenario above. If it's a legitimate distribution shift the baseline constraints file hasn't been updated to reflect, the fix is updating the baseline — but that update itself should go through review, since loosening a data-quality check is exactly the kind of change that needs a second set of eyes in a regulated environment, not a quiet one-line config edit.

SCENARIO

☐ mastered

The fraud model starts blocking a disproportionate number of legitimate transactions — false-positive spike.

This one has a customer-trust dimension a normal latency incident doesn't, so speed of communication matters alongside speed of fix. Confirm the spike is real (not a reporting/dashboard artifact) and quantify it — how many customers, how much value, over what window; the business will ask this within minutes. Check whether it correlates with a recent deploy (rollback is the fast lever if so) or an upstream data issue (as in scn2/scn5). If it's a genuine model behavior issue with no fast root cause, the containment decision is a business one, not purely technical: is the false-positive rate bad enough to justify falling back to a more conservative/manual-review-heavy mode while you investigate, accepting some fraud may slip

through temporarily. That trade-off decision should follow a pre-agreed escalation path, not be made solo.

SCENARIO

mastered

A DR failover drill reveals your actual RPO is worse than what's documented.

Treat the drill as having done its job — this is exactly why you test instead of trusting the architecture diagram. Quantify the actual gap (measured replication lag vs. documented RPO) and find the specific cause: a replication process that's slower than assumed, a component that wasn't actually included in the replication scope, or a documented RPO that was never validated in the first place and was simply optimistic. Report it honestly rather than quietly patching the number in the DR doc to match reality — a corrected RPO with a remediation plan and timeline is a much better audit finding than a stale document that turns out to be wrong during an actual disaster. Then fix the underlying gap and re-test before calling it closed.

SCENARIO

mastered

You discover a model version reached production without going through the approval gate.

Treat it as a compliance incident, not just a process hiccup — report it up immediately rather than quietly fixing it and moving on, because in a regulated environment an unapproved model having served live decisions is exactly the kind of thing that needs to be in the incident record even if the model itself turned out to be fine. Determine the exposure window and what decisions it made. Find the control gap that let it happen — a pipeline misconfiguration that skipped the gate, a manual deploy that bypassed CI/CD entirely, or a gate that exists on paper but isn't actually technically enforced (see §6's segregation-of-duties question). Fix the control, not just this one instance, and make sure the fix is verified to actually block the same bypass next time.

SCENARIO

mastered

Overnight cost alert: a SageMaker endpoint's spend tripled with no obvious traffic change.

Check instance count first — a stuck or misfiring autoscaling policy scaling out and never back down is the most common cause, sometimes triggered by a noisy scaling metric rather than real load. Check whether a deploy changed the instance type to something larger without an

accompanying reason. Rule out a duplicate/orphaned endpoint left running from a test or a previous deployment that never got cleaned up — easy to miss and a surprisingly common source of "mystery" cost. Once identified, fix the immediate cost bleed, then look at whether better scaling-policy tuning or a scheduled scale-down for predictable low-traffic windows would prevent a repeat, rather than treating this as a one-off.

13 · BEHAVIORAL, POSITIONING & CLOSING

Behavioral, Positioning & Closing Questions

Frameworks to fill in with your own real examples — not scripted stories. The two positioning items (§2–3) are the ones most worth rehearsing word-for-word, since they're the exact gaps a sharp interviewer will probe.

FRAME "Walk me through your experience with AI/ML production support."

☐ mastered

Three moves, in order: **(1) Ground in what's real** — 14+ years in SRE/DevOps, deep in incident management, Kubernetes, CI/CD, and AWS operations, including regulated-banking environments (Barclays, Russell Investments) where compliance and change control weren't optional. **(2) Name the transfer explicitly** — production support for an ML system is the same discipline (observability, incident response, safe deployment, DR) applied to a system with an extra failure mode: it can be "up" and still wrong. That's a mindset shift you understand, not a skill you're missing. **(3) Be precise about what's genuinely new** — the ML-specific tooling (SageMaker Pipelines, drift monitoring, Model Registry workflows) is where you're actively building depth, and say specifically how (this handbook, hands-on exploration, whatever's true for you) rather than leaving it vague. Precision about the boundary between "solid" and "growing" reads as more senior than claiming uniform expertise.

FRAME "Did you design this, or did the platform team?"

☐ mastered

This exact question pattern showed up repeatedly in your Aug 5 AI/ML-in-banking interview — across Terraform architecture, IAM, networking, and security — and "the platform team handled that" alone reads as weaker than it should. The tested reframe: **"the platform/architecture team owned the design decisions; my role was operating, troubleshooting, and evolving what they built"** — then immediately follow with a specific example of something you *did* change or improve within that system, even a modest one. This is honest (you're not claiming design authorship you don't have) while showing you weren't passive inside the system either.

Have this phrasing ready before the interview starts, not improvised in the moment when the question catches you off guard.

FRAME **"Your certifications are Azure — this role is heavily AWS. Walk me through that."** mastered

Don't get defensive or over-apologize — state it plainly and pivot to substance fast: *"My certifications happen to be Azure — AZ-104, SC-300, SC-100 — but my hands-on production experience is AWS-heavy, about six years across EC2, EKS, Lambda, and S3 in the environments I've supported. The identity and security-architecture thinking behind SC-300/SC-100 — least-privilege design, governance, security-by-design — is exactly the same thinking IAM/KMS/Secrets Manager work needs on AWS; the concepts transfer, only the console and CLI syntax differ, and that's a fast switch for someone who's been hands-on across both."* Said once, confidently, this closes the question — over-explaining it signals more insecurity about the gap than the gap itself would.

STAR **"Tell me about a production incident you managed end-to-end."** mastered

Pick your strongest *real* incident and structure it tightly: **Situation** — what broke, what was the blast radius, how did you find out. **Task** — what was actually your responsibility in the response (be honest if you were part of a team, not the sole responder). **Action** — the concrete sequence: contain first, then diagnose, then fix, then RCA — naming specific tools/commands you actually used. **Result** — quantified if you genuinely have the numbers (downtime avoided, MTTR, recurrence prevented), qualified honestly if you don't ("this hasn't recurred since" is fine without a fabricated percentage). Close with the process change that came out of it — interviewers weight "did this change how the team operates" as much as the fix itself. Use a real one; a vague or invented incident falls apart fast under natural follow-up questions like "what did the dashboard actually show you."

FRAME **"How do you balance regulatory/compliance requirements against operational speed?"** mastered

Resist the framing that these are opposites you trade off case-by-case — the stronger answer is that good automation makes them the same thing. A pipeline with a technically-enforced approval gate is both faster (no manual chasing-down-a-signature delay) and more compliant (unbypassable, logged) than a slow manual process that's also weakly enforced. Where a genuine trade-off exists — an emergency rollback that needs to happen faster than the normal

change process allows — the right answer is a pre-agreed emergency-change path with retroactive review, not "I'd skip the process this once" and not "I'd always wait for full sign-off even during an active incident." Naming that pre-agreed middle path is what signals real experience with regulated environments.

FRAME **"Explain a model drift alert to a non-technical risk/compliance stakeholder."** ☐ mastered

Skip PSI/KS-test vocabulary entirely and lead with business impact: *"The pattern of transactions the model is seeing today looks meaningfully different from what it was trained on — which means its decisions are less reliable than usual until we confirm what changed. We caught this automatically before it caused a real problem, we're treating it as [routine monitoring / an active investigation], and here's what happens next and by when."* The stakeholder doesn't need the statistical test; they need to know whether this is a "watching it" situation or a "stop and look" situation, what the exposure is, and when they'll hear back. Translating technical certainty into business-risk language, clearly and without jargon, is itself the skill being tested in this question.

FRAME **"Why DataArt?"** ☐ mastered

Something genuine to point to rather than generic flattery: DataArt has a real, decades-deep financial-services practice (banks, capital markets, payment providers, fraud-detection systems) and is positioned as an AWS Consulting Partner specifically for that sector — which matches this JD's stack directly. Their public positioning around "Artisyn," their AI-delivery model, explicitly talks about embedding governance and human oversight into AI work for regulated clients rather than treating compliance as an afterthought — that's a good hook if you genuinely care about the governance side of this role (which, given the JD, you should be prepared to say you do). Keep it short and specific rather than reciting the company's own marketing back at them wholesale.

CLOSER **Good questions to ask the interviewer.** ☐ mastered

- "Is the ML-serving stack split between SageMaker endpoints and EKS today, or moving toward one over the other?" — shows you understood the JD's dual emphasis and care about the real architecture, not just the buzzword list.
- "Which registry — SageMaker's or MLflow's — is the actual system of record here?" — a genuinely useful operational question (see §3) that also signals depth.
- "Which banking client(s) or business line does this role primarily support, and what's already in production versus greenfield?" — helps you calibrate whether you'd be

maintaining or building.

- "What does on-call actually look like for this team — rotation size, page volume, typical severity mix?" — a fair, practical question that also signals you're evaluating fit, not just trying to get an offer.
- "How does DataArt's Artisyn governance model show up day-to-day for someone in this seat?" — ties directly to their own stated positioning; shows you did real homework.

CLOSER **Closing framing for the whole interview.** ☐ mastered

This JD is, underneath the "AI" framing, squarely your archetype — production support/SRE discipline applied to a new class of system, in a domain (banking, regulated, AWS-heavy) where your real background (Barclays, Russell Investments, 14+ years, AWS 6+ years) is directly on point. The honest gaps are narrow and nameable: hands-on depth with the specific ML-tooling layer (SageMaker Pipelines/MLflow mechanics) and the Azure-certs-vs-AWS-role optics — both covered above, both closable in a single confident sentence each rather than something to over-explain. Go in leading with strength, name the gaps once and move on, and don't let the "AI" in the title talk you into underselling fifteen years of exactly the operational judgment this role is actually testing for.